# Lab No. 01 — C Warm-up for Parallel Programming (Part 1)

Dr. Rochdi Boudjehem · 2026-10-26

*Without solutions*

**Duration:** 1 session (Exercises 0 to 4). Lab 2 continues the warm-up with Exercises 5 to 9.

**Why this lab?** Every program you will write in the next labs (MPI, PThreads, OpenMP, CUDA, OpenCL) is a C program. They are all compiled in several steps, often through shell scripts; they read their parameters from the command line (`argc`, `argv`); and they use the preprocessor (`#include`, `#define`). This lab reviews these tools so that, in the next labs, you can focus on parallelism and not on C. Lab 2 then covers processes, memory, matrices, function pointers, timing and Makefiles.

### Rules for the whole lab

- Always compile with warnings on: `gcc -Wall -Wextra`. A warning is a bug that has not happened yet.
- Work in the folder `~/hpc/lab01`, and keep your files.
- Most exercises end with a **script** `exNN.sh` (Exercise 1, Part B shows how to write one) that compiles and runs the programs of the exercise. Re-running an exercise is then a single command, e.g. `./ex02.sh`.
- For each exercise, write the answers to the questions in a file `answers.txt`.

### Resources on Moodle

The course page https://elearning.univ-guelma.dz/course/view.php?id=16843, section *(TP): Practical Work*, has the file `lab01_resources.zip`, so you do not have to copy code from this PDF:

- `install.sh`: installs all the tools of the semester (Exercise 0) and creates the folders `~/hpc/lab01` to `~/hpc/lab07`. Run it **once, at home, before this lab**: it downloads several hundred MB.
- `fix-mpirun-version.sh`: fixes the error message of `mpirun --version` if it occurs (Exercise 0, question 3).
- `hello.c`, `macros.c`: the programs given in Exercises 1 and 4.
- `bugs.c`, `Makefile`: the files of Lab 2 (Exercise 9).

> The code in the boxes of this PDF can also be copied directly: select it and copy it (Ctrl+C). The copy keeps the indentation and leaves out the line numbers and the prompts (`me@PCinfo:~/hpc/lab01$`).

Download the zip in Windows. Then open the Windows File Explorer and go to your Linux home folder: either click the penguin icon **Linux** in the left panel, then double-click `Ubuntu`, `home` and your Linux user folder (mine is `me`), or type `\\wsl.localhost\Ubuntu\home\<your_user>` in the address bar.

Create a folder `hpc` there (right-click > New > Folder, or Ctrl+Shift+N) and copy the zip into it. Then, in a WSL terminal, run the following commands:

```
me@PCinfo:
```

```
me@PCinfo:~$ cd ~/hpc
me@PCinfo:~/hpc$ sudo apt install -y unzip
me@PCinfo:~/hpc$ unzip lab01_resources.zip
me@PCinfo:~/hpc$ cd lab01_resources
me@PCinfo:~/hpc/lab01_resources$ chmod +x *.sh
me@PCinfo:~/hpc/lab01_resources$ ./install.sh
me@PCinfo:~/hpc/lab01_resources$ cp hello.c macros.c ~/hpc/lab01/
me@PCinfo:~/hpc/lab01_resources$ cd ~/hpc/lab01
me@PCinfo:~/hpc/lab01$ ls
hello.c  macros.c
```

Work in your Linux folder `~/hpc`, never in `/mnt/c/...`. Open the `.sh` files and the `Makefile` with VS Code connected to WSL, not with Notepad: Notepad can change the line endings of the scripts (error `/bin/bash^M: bad interpreter`) and the tab of the `Makefile` (error `missing separator`).

# Exercises

## Exercise 0. The Working Environment (10 min)

We work under Linux. On Windows, we use **WSL** (Windows Subsystem for Linux) with Ubuntu, and VS Code connected to WSL.

**Have you already downloaded `lab01_resources.zip` from Moodle and run `install.sh`?**

> If you followed the steps of *Resources on Moodle* above: the tools are installed and the folders `~/hpc/lab01` to `~/hpc/lab07` exist.
> Skip question 1 and go directly to question 2 (read the commands of question 1 to answer it), then check the versions (question 3).

1. If you could not get `lab01_resources.zip`, install the packages manually. On Linux, open a terminal (`bash` is recommended); on Windows 10 or 11, open a WSL terminal (or the VS Code terminal connected to WSL). Then install all the tools of the semester with these commands (they are the content of `install.sh`):

```
me@PCinfo:
```

```
me@PCinfo:~/hpc/lab01$ sudo apt update && sudo apt full-upgrade -y
me@PCinfo:~/hpc/lab01$ sudo apt install -y build-essential gcc g++ make cmake git xxd
me@PCinfo:~/hpc/lab01$ sudo apt install -y openmpi-bin libopenmpi-dev libomp-dev
me@PCinfo:~/hpc/lab01$ sudo apt install -y ocl-icd-opencl-dev opencl-headers
pocl-opencl-icd clinfo
```

2. For each package (or group of packages) in these commands, say which lab will use it.

3. Check the installation:

```
me@PCinfo:
```

```
me@PCinfo:~/hpc/lab01$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

me@PCinfo:~/hpc/lab01$ mpicc --version
mpicc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

me@PCinfo:~/hpc/lab01$ mpirun --version
mpirun (Open MPI) 5.0.10

Report bugs to https://www.open-mpi.org/community/help/

me@PCinfo:~/hpc/lab01$ clinfo | head -5
Number of platforms                               1
  Platform Name                                   Portable Computing Language
  Platform Vendor                                 The pocl project
  Platform Version                                OpenCL 3.0 pocl 3.0, LLVM 17.0.6,
RELOC, SLEEF, DISTRO, POCL_DEBUG
  Platform Profile                                FULL_PROFILE
```

If `mpirun --version` prints the following error:

```
me@PCinfo:
```

```
me@PCinfo:~/hpc/lab01$ mpirun --version
Sorry!  You were supposed to get help about:
    version
```

```
But I couldn't open the help file:
    /usr/lib/x86_64-linux-gnu/pmix2/share/pmix/help-schizo-ompi.txt: No such file or
directory
    /usr/lib/x86_64-linux-gnu/prrte3/share/prte/help-schizo-ompi.txt: No such file or
directory.
Sorry!
```

The bug only breaks the help and error messages. Running programs with `mpirun -np 4 ./prog` should still work, but when something fails you would get this same unhelpful "Sorry!" text. It is worth fixing.

To fix it, run these commands (or use `fix-mpirun-version.sh` included in the zip):

me@PCinfo:

```
me@PCinfo:~/hpc/lab01$ sudo mkdir -p /usr/lib/x86_64-linux-gnu/prrte3/share/prte
me@PCinfo:~/hpc/lab01$ sudo cp /usr/share/doc/libprrte-dev/help-*.txt*
/usr/lib/x86_64-linux-gnu/prrte3/share/prte/
me@PCinfo:~/hpc/lab01$ sudo gunzip -f /usr/lib/x86_64-linux-gnu/prrte3/share/prte/*.gz
me@PCinfo:~/hpc/lab01$ mpirun --version
```

The last command should now print the version and the copyright.

4. If you have already created your lab folder, open it in VS Code. Otherwise, create your lab folder and open it in VS Code:

me@PCinfo:

```
me@PCinfo:~$ mkdir -p ~/hpc/lab01
me@PCinfo:~$ cd ~/hpc/lab01
me@PCinfo:~/hpc/lab01$ code .
```

> **Tip.** If you are on Windows, `code .` launched from a WSL (Ubuntu) terminal opens the Windows version of VS Code, connected to WSL.
> If you are on Linux, it opens VS Code normally.
> In both cases, the terminal is a Linux terminal.

## Exercise 1. From Source Code to Executable, and Shell Scripts (25 min)

### Part A. The four steps of compilation

In the `~/hpc/lab01` folder, open `hello.c` in VS Code. If you do not have it, create a file `hello.c` with the following content.

**hello.c**

```c
#include <stdio.h>
#define GREETING "Hello, HPC!"

int main(void) {
    printf("%s\n", GREETING);
    return 0;
}
```

1. Run the four steps of the compilation one by one, and look at the size and the content of each file produced (`ls -l`, `file`, `wc`, `head`, `xxd`, `nm`). Compare your output with the one below.

#### Step 1. Preprocessing

me@PCinfo:

```
me@PCinfo:~/hpc/lab01$ gcc -E hello.c -o hello.i
me@PCinfo:~/hpc/lab01$ ls -l hello.c hello.i
```

```
-rw-r--r-- 1 me me   114 Nov  2 09:22 hello.c
-rw-r--r-- 1 me me 21444 Nov  2 11:22 hello.i   # size = x188

me@PCinfo:~/hpc/lab01$ file hello.i
hello.i: C source, ASCII text

me@PCinfo:~/hpc/lab01$ wc -l hello.c hello.i
    7 hello.c
  845 hello.i
  852 total

me@PCinfo:~/hpc/lab01$ head -n 12 hello.i
# 0 "hello.c"
# 0 "<built-in>"
# 0 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 0 "<command-line>" 2
# 1 "hello.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 28 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 431 "/usr/include/features.h" 3 4
```

2. Search for `GREETING` in `hello.c` and `hello.i` using the `grep` command.
3. What happened to it? Check the last four lines of `hello.i` with `tail -n 4 hello.i`. Compare them with the last four lines of `hello.c`.

```
me@PCinfo:

me@PCinfo:~/hpc/lab01$ grep GREETING hello.c
#define GREETING "Hello, HPC!"
    printf("%s\n", GREETING);
me@PCinfo:~/hpc/lab01$ tail -n 4 hello.c
int main(void) {
    printf("%s\n", GREETING);
    return 0;
}

me@PCinfo:~/hpc/lab01$ grep GREETING hello.i
me@PCinfo:~/hpc/lab01$ tail -n 4 hello.i
int main(void) {
    printf("%s\n", "Hello, HPC!");
    return 0;
}
```

4. Open `hello.i` in VS Code and look at its content. Why is `hello.i` so long?

**Step 2. Compilation to assembly**

```
me@PCinfo:

me@PCinfo:~/hpc/lab01$ gcc -S hello.i -o hello.s

me@PCinfo:~/hpc/lab01$ ls -l hello.s
-rw-r--r-- 1 me me 659 Nov  2 11:22 hello.s

me@PCinfo:~/hpc/lab01$ file hello.s
hello.s: assembler source, ASCII text

me@PCinfo:~/hpc/lab01$ head -n 25 hello.s
```

```
    .file   "hello.c"
    .text
    .section    .rodata
.LC0:
    .string "Hello, HPC!"
    .text
    .globl  main
    .type   main, @function
main:
.LFB0:
    .cfi_startproc
    endbr64
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    leaq    .LC0(%rip), %rax
    movq    %rax, %rdi
    call    puts@PLT
    movl    $0, %eax
    popq    %rbp
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
```

**Step 3. Assembly to an object file**

me@PCinfo:

```
me@PCinfo:~/hpc/lab01$ gcc -c hello.s -o hello.o
me@PCinfo:~/hpc/lab01$ ls -l hello.o
-rw-r--r-- 1 me me 1496 Nov  2 11:22 hello.o

me@PCinfo:~/hpc/lab01$ file hello.o
hello.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped

me@PCinfo:~/hpc/lab01$ head -c 64 hello.o | xxd
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF............
00000010: 0100 3e00 0100 0000 0000 0000 0000 0000  ..>.............
00000020: 0000 0000 0000 0000 5802 0000 0000 0000  ........X.......
00000030: 0000 0000 4000 0000 0000 4000 0e00 0d00  ....@.....@.....

me@PCinfo:~/hpc/lab01$ nm hello.o
0000000000000000 T main
                 U puts
```

**Step 4. Linking (dynamic linking with the C library)**

me@PCinfo:

```
me@PCinfo:~/hpc/lab01$ gcc hello.o -o hello
me@PCinfo:~/hpc/lab01$ ls -l hello
-rwxr-xr-x 1 me me 15952 Nov  2 11:22 hello

me@PCinfo:~/hpc/lab01$ file hello
hello: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=2b7462671f0b2ba24d828e3cd0f804ece6b3a268, for GNU/Linux 3.2.0, not
stripped

me@PCinfo:~/hpc/lab01$ head -c 64 hello | xxd
```

```
00000000: 7f45 4c46 0201 0100 0000 0000 0000 0000  .ELF............
00000010: 0300 3e00 0100 0000 6010 0000 0000 0000  ..>.....`.......
00000020: 4000 0000 0000 0000 9036 0000 0000 0000  @........6......
00000030: 0000 0000 4000 3800 0e00 4000 1f00 1e00  ....@.8...@.....

me@PCinfo:~/hpc/lab01$ nm hello | grep -E ' (main|puts)'
0000000000001149 T main
                 U puts@GLIBC_2.2.5

me@PCinfo:~/hpc/lab01$ ./hello
Hello, HPC!
```

5. What is the role of the linker? Which library does it add here?
6. What is the difference between `gcc hello.c` and `gcc hello.c -o hello`? Why do we write `./hello` and not `hello`?
7. Later, you will compile with `mpicc`, `gcc -fopenmp` and `gcc ... -lOpenCL`. Run `mpicc --showme hello.c -o hello`. What does `mpicc` really do?

### Part B. Shell scripts to compile and run

Typing the same commands again and again is slow and error-prone. A **shell script** is a text file (extension `.sh`) that contains a list of shell commands, executed one after the other, exactly as if you typed them.

8. Create an empty script file with `touch` and look at it with `ls -l`:

```
me@PCinfo:

me@PCinfo:~/hpc/lab01$ touch ex01.sh
me@PCinfo:~/hpc/lab01$ ls -l ex01.sh
```

9. Type the following content into the script, or copy it from this PDF and paste it, with a terminal editor (vim, nano or emacs) or, more simply, with VS Code:

**ex01.sh**

```
#!/bin/bash
# Exercise 1: the four steps of the compilation of hello.c
set -e                          # stop at the first command that fails

gcc -E hello.c -o hello.i                  # 1. preprocessing
gcc -Wall -Wextra -S hello.i -o hello.s    # 2. compilation to assembly (with
    ↪ warnings)
gcc -c hello.s -o hello.o                  # 3. assembly to object file
gcc hello.o -o hello                       # 4. linking

ls -l hello.c hello.i hello.s hello.o hello
./hello
```

9.a **Using `vim`** (installed by default in Ubuntu, WSL included; otherwise `sudo apt install vim`):

```
me@PCinfo:

me@PCinfo:~/hpc/lab01$ vim ex01.sh
```

- Press `i` to enter insert mode.
- Paste the content of the script (`Ctrl+V` or **right-click** in WSL, some linux distros support `Ctrl+Shift+V`).
- Press `Esc` to leave insert mode (important: Vim accepts commands only outside insert mode).
- Type `:wq` and press `Enter` to save and quit. (In Vim, `:` starts a command; it is shown on the bottom line of the screen and runs when you press `Enter`.)

9.b **Using `nano`** (installed by default in Ubuntu, WSL included; otherwise `sudo apt install nano`):

```
me@PCinfo:
```
```
me@PCinfo:~/hpc/lab01$ nano ex01.sh
```

- Paste the content of the script (`Ctrl+V` or **right-click** in WSL, some linux distros support `Ctrl+Shift+V`).
- Press `Ctrl+O`, then `Enter` to confirm the file name.
- Press `Ctrl+X` to exit nano.

9.c **Using `emacs`** (not installed by default; install it with `sudo apt install emacs`):

```
me@PCinfo:
```
```
me@PCinfo:~/hpc/lab01$ emacs ex01.sh
```

- Paste the content of the script (`Ctrl+V` or **right-click** in WSL, some linux distros support `Ctrl+Shift+V`).
- Press `Ctrl+X Ctrl+S` successively to save the file.
- Press `Ctrl+X Ctrl+C` successively to exit emacs.

9.d **Using VS Code**

- Open the script in VS Code: `code ex01.sh`.
- Paste the content of the script.
- Save the file (`Ctrl+S`).

> **Tip.** VS Code is the easiest way to edit files, so why learn these tedious terminal editors?
>
> Desktop Windows and Linux are mouse-friendly, but sometimes the terminal is all you have: on a remote server such as an HPC cluster, reached through `ssh`, or on a system that no longer boots, where you must edit system files.
>
> **Example 1:** One of the most common boot errors comes from a missing or erroneous GRUB configuration file.
>
> To fix it, you have to edit the file `/etc/default/grub` with `sudo vim /etc/default/grub` or `sudo nano /etc/default/grub`, then run `sudo update-grub`. If you are not familiar with these editors, you will be lost.
>
> **Example 2:** Another common boot error comes from a wrong `/etc/fstab`, the small text file that lists the partitions to mount at startup.
>
> To fix it, you have to edit it with `sudo vim /etc/fstab` or `sudo nano /etc/fstab`. If you are not familiar with these editors, you will be lost.

10. Display the content of the script with `cat ex01.sh`. Then try to run it:

```
me@PCinfo:
```
```
me@PCinfo:~/hpc/lab01$ cat ex01.sh
#!/bin/bash
# Exercise 1: the four steps of the compilation of hello.c
set -e                          # stop at the first command that fails

gcc -E hello.c -o hello.i                   # 1. preprocessing
gcc -Wall -Wextra -S hello.i -o hello.s     # 2. compilation to assembly (with
warnings)
gcc -c hello.s -o hello.o                   # 3. assembly to object file
gcc hello.o -o hello                        # 4. linking

ls -l hello.c hello.i hello.s hello.o hello
./hello
me@PCinfo:~/hpc/lab01$ ./ex01.sh
```

What error do you get? Look at the output of `ls -l ex01.sh` again.

```
me@PCinfo:
```
```
me@PCinfo:~/hpc/lab01$ ls -l ex01.sh
-rw-r--r-- 1 me me 473 Nov  2 12:15 ex01.sh
```

11. Give the **execute** permission to the script, check the permissions, and run it:

```
me@PCinfo:
```

```
me@PCinfo:~/hpc/lab01$ chmod +x ex01.sh
me@PCinfo:~/hpc/lab01$ ls -l ex01.sh
-rwxr-xr-x 1 me me 473 Nov  2 12:15 ex01.sh
me@PCinfo:~/hpc/lab01$ ./ex01.sh
```

12. Explain the 10 characters `-rwxr-xr-x` shown by `ls -l`. What did `chmod +x` change? The same result is obtained with `chmod 755 ex01.sh`: explain the number `755`.
13. A script can also be run with `bash ex01.sh`, even without the execute permission. Why?
14. What is the role of the first line `#!/bin/bash` (the *shebang*)? And of the lines that start with `#`?

If running the script prints this error:

```
me@PCinfo:
```

```
me@PCinfo:~/hpc/lab01$ ./ex01.sh
bash: ./ex01.sh: /bin/bash^M: bad interpreter: No such file or directory
```

> **Tip.** The file was saved with Windows line endings (CRLF). Fix it with `sed -i 's/\r$//' ex01.sh`, or click `CRLF` in the VS Code status bar and choose `LF`.

> **Tip.** Always work in your Linux home folder (`~/hpc/...`), not in a Windows folder such as `/mnt/c/...` or `/mnt/d/...`: there, every file appears as `-rwxrwxrwx` and `chmod` has no effect.

## Exercise 2. Command-Line Arguments: `argc` and `argv` (20 min)

When the shell starts a program, it passes it the words of the command line. `main` receives them through two parameters:

```c
int main(int argc, char *argv[])   /* also written: char **argv */
```

1. Write `args_test.c` that prints `argc`, then each `argv[i]` between quotes, one per line.

```c
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("argc = %d\n", argc);
    for (int i = 0; i < argc; i++) {
        printf("argv[%d] = \"%s\"\n", i, argv[i]);
    }
    return 0;
}
```

2. Compile it with `gcc -Wall -Wextra args_test.c -o args_test`.
3. Run it with:

```
me@PCinfo:
```

```
me@PCinfo:~/hpc/lab01$ ./args_test
me@PCinfo:~/hpc/lab01$ ./args_test hello 42 3.14
me@PCinfo:~/hpc/lab01$ ./args_test "two words" 'single quotes' two words
me@PCinfo:~/hpc/lab01$ ./args_test *.c
```

For each run, explain the value of `argc` and the content of `argv[0]`.

4. What is the type of `argv[1]` when you type `./args_test 42`? Can you write `argv[1] + 1` to get 43? Try it and explain the result.

To test it, add this fragment at the end of `main`, before `return 0;`:

```
    if (argc > 1) {
        printf("argv[1] = \"%s\"\n", argv[1]);
        printf("argv[1] + 1 = \"%s\"\n", argv[1] + 1);
    }
```

5. What is the value of `argv[argc]`? Check it in your program.
6. Add the line `printf("%s\n", argv[1]);` at the end of `main` and run `./args_test` with no argument. What happens? Conclusion: what must **always** be checked before using `argv[i]`?
7. In Lab 3, every MPI program starts with `MPI_Init(&argc, &argv);`. Why does MPI need access to `argc` and `argv`?

**Script.** Write `ex02.sh`, which compiles `args_test.c` with `-Wall -Wextra` and runs the four commands of question 3. Use `echo "=== ./args_test hello 42 3.14"` before each run to label the output. Make it executable and run it.

## Exercise 3. Converting Arguments and Exit Codes (25 min)

### Part A. `atoi` vs `strtol`

1. Write a small test that prints `atoi("42")`, `atoi("0")`, `atoi("abc")`, `atoi("12abc")` and `atoi("")`. Why is `atoi` dangerous for a program that receives its parameters from a user?
2. Read `man strtol`. Explain the role of each of its three parameters. In the lecture (Chapter 2), the number of threads is read with `strtol(argv[1], NULL, 10)`. What does `NULL` mean here?

> **Tip.** `man` stands for *manual*. It is a command that displays the manual of a command or a function. For example, `man strtol` shows the manual of the function `strtol`. Use `q` to quit the manual.

> **Tip.** `atoi` stands for **a**scii **to** **i**nteger. It is a simpler function than `strtol`, but it does not detect errors.

> **Tip.** `strtol` stands for **str**ing **to** **l**ong. It can handle different bases (decimal, hexadecimal, octal).

## Exercise 4. The Preprocessor: `#define`, `#ifdef` and Predefined Macros (20 min)

Write `macros.c`:

```
#include <stdio.h>

#ifndef N
#define N 10
#endif

#define SQUARE(x)    x * x
#define SQUARE_OK(x) ((x) * (x))

int main(void) {
    printf("SQUARE(N + 1)    = %d\n", SQUARE(N + 1));
    printf("SQUARE_OK(N + 1) = %d\n", SQUARE_OK(N + 1));
#ifdef DEBUG
    printf("[debug] N = %d, file %s, line %d\n", N, __FILE__, __LINE__);
#endif
    return 0;
}
```

1. Predict the two printed values, then run the program to check your prediction. Use `gcc -E macros.c | tail` to explain the difference.
2. Compile with `gcc -Wall -DDEBUG -DN=20 macros.c -o macros`. What does the option `-D` do? What is the role of `#ifndef N`?

3. Give one advantage of a value fixed with `-DN=...` at compile time, and one advantage of a value read from `argv` at run time.
4. What do `__FILE__` and `__LINE__` give in the debug line? They are not variables: what are they? Why do their names start and end with two underscores?

> **Tip.** `__FILE__` and `__LINE__` are **predefined macros**: the compiler defines them itself, without any `#define`.
>
> The C standard also defines `__DATE__`, `__TIME__` and `__STDC_VERSION__`; each compiler adds its own, such as `__GNUC__` for gcc, and OpenMP adds `_OPENMP`.
>
> Names that start with two underscores (`__` also called "double underscores", shortly "dunders") are **reserved** for the compiler and the system, so they never collide with your own names: never start your names with `__`.
>
> You will meet more of them in Labs 5 and 6: CUDA marks a function that runs on the GPU with `__global__` or `__device__`, and OpenCL marks a kernel with `__kernel`.
>
> These are not macros but **qualifiers**, keywords that tell the compiler where the code runs.

## Self-Check

At the end of this lab, you should be able to answer **yes** to each question:

- I can check the tools of the semester (`gcc`, `mpicc`, `mpirun`, `clinfo`) and open my lab folder in VS Code from WSL.
- I can run the four steps of the compilation, and I know what `-E`, `-S`, `-c`, `-o` and `-Wall -Wextra` do.
- I can create a script with `touch`, edit it, display it with `cat`, make it executable with `chmod +x`, and read the permissions shown by `ls -l`.
- I always check `argc` before using `argv[i]`, and I know why `atoi` is unsafe and how `strtol` reports errors.
- I can define a macro with `#define` or `-D`, I know why its parameters need parentheses, and I know what `__FILE__` and `__LINE__` are.